PYTHON PROGRAMMING SERIES

Web Scraping with BeautifulSoup

Learn how to automate web data collection using Python's powerful libraries: Requests & BeautifulSoup

1. Introduction: Manual vs. Automated Data Collection

Imagine you are tracking live scores for your favorite sports tournament, such as IPL or ISL. Copying and pasting scores manually line-by-line is slow, tedious, and prone to human error. But can a program do this task for you?

Manual Collection

Copy-pasting every single score or piece of text by hand takes hours, causes visual fatigue, and requires constant manual monitoring.

Automated Scraping

A smart Python script collects, structures, and saves thousands of rows of data automatically in just seconds with 100% precision!

2. What Is Web Scraping?

Web scraping means writing a computer program that automatically reads web pages and extracts specific information you need.

> **Analogy:** Think of web scraping like a food delivery driver who picks up lunchboxes from different locations and delivers them straight to your home. Similarly, a scraping script extracts scattered data across web pages and brings it directly to your program!

3. Spotting HTML Tags

HTML Tag	Meaning / Description	Example Use Case
`<h1>`	Big Heading	Main title of an article or webpage
`<ul>`	Unordered List	A bulleted list container
`<li>`	List Item	An individual item inside a list
`<a>`	Anchor / Link	Hyperlinks to other web pages or assets
`<table>`	Data Table	Structured rows and columns of data

4. Core Tools: Requests & BeautifulSoup

- **Requests:** Acts like the postman bringing the HTML webpage code from the internet straight to your Python program.
- **BeautifulSoup:** Acts like the detective parsing (reading) the raw HTML markup and finding specific elements or content.

5. Core Code Examples

Example A: Fetching & Parsing Basic Data

```
import requests from bs4 import BeautifulSoup # Step 1: Request page contents from web server url = "https://example.com/india-national-parks" response = requests.get(url) # Step 2: Parse HTML content with BeautifulSoup soup = BeautifulSoup(response.text, "html.parser") # Step 3: Extract main headline and top 5 national parks headline = soup.find("h1").text parks = soup.find_all("li") [:5] print("Headline:", headline) for park in parks: print("-", park.text)
```

Example B: Scraping HTML Attributes (Extracting Hyperlinks)

```
from bs4 import BeautifulSoup html_doc = ''' <div class="sports-news"> <a href="https:// example.com/match1" class="link">Match 1 Details</a> <a href="https://example.com/match2" class="link">Match 2 Details</a> </div> ''' soup = BeautifulSoup(html_doc, "html.parser") links = soup.find_all("a") for link in links: title = link.text url = link['href'] # Access attribute like a dictionary key print(f"Title: {title} | URL: {url}")
```

Example C: Filtering by CSS Class and Parent-Child Navigation

```
from bs4 import BeautifulSoup html_table = ''' <tr class="score-row"> <td class="team">Mumbai Indians</td> <td class="score">185/4</td> </tr> ''' soup = BeautifulSoup(html_table, "html.parser") team = soup.find("td", class_="team").text score = soup.find("td", class_="score").text print(f"Team: {team} scored {score}")
```

6. Finding Elements: find() vs. find_all()

Method	Behavior	Analogy	Code Example
`find()`	Returns the FIRST matching element.	Picking the first mango from a basket	`soup.find("h1")`
`find_all()`	Returns a list of ALL matching elements.	Collecting ALL mangoes at once	`soup.find_all("li")`

7. Ethical Web Scraping Rules

- **Check Permissions:** Inspect the site's `robots.txt` file (e.g., `example.com/robots.txt`) to check scraping rules.
- **Don't Overload Servers:** Add delays using `time.sleep()` to prevent server crashes.
- **Use Data Responsibly:** Extract and utilize data only for legal, educational, and constructive purposes.
- **Respect Privacy:** Never attempt to scrape personal or confidential user data.

PRACTICE WORKSHEET & STUDENT ASSESSMENT

Question 1: Conceptual Knowledge

Explain the difference between manual data collection and automated web scraping. Why is web scraping preferred for large datasets?

Question 2: Tool Roles

Describe the specific roles of the `requests` library and the `BeautifulSoup` library in a Python web scraping script.

Question 3: HTML Tag Matching

Match the following HTML tags to their correct description:

a) `<h1>` b) `<ul>` c) `<li>` d) `<a>`

Question 4: BeautifulSoup Methods

Consider a webpage containing multiple paragraph (`<p>`) tags:

1. What method would you use if you only want to retrieve the very first paragraph?
2. What method would you use if you want to extract every paragraph on the page into a list?

Question 5: Practical Code Implementation (Extracting Attributes)

Write a complete Python code snippet using BeautifulSoup to parse the following HTML string and print all hyperlink URLs:

```
html_data = '<div><a href="https://site1.com">One</a><a href="https://site2.com">Two</a></
div>'
```

Question 6: Code Analysis & Output Prediction

Given the following Python code snippet, write down what output will be printed:

```
import requests from bs4 import BeautifulSoup html_data = "<ul><li>Apple</li><li>Banana</
li><li>Cherry</li></ul>" soup = BeautifulSoup(html_data, "html.parser") items =
soup.find_all("li") for item in items: print(item.text)
```

Question 7: Table Scraping Challenge

Write code using `find_all('td')` to extract and print text from all table cells in an HTML table string.

Question 8: Ethics in Scraping

List three ethical rules or best practices you must follow before and while web scraping a live website.